

NoteNest.git

SYSTEM PURPOSE: NoteNest is a web application designed to store and manage notes. It allows users to create, edit, and delete notes, as well as upload files. The system is intended to provide a simple and user-friendly interface for note-taking and organization.

TECH STACK: - backend: Python - frontend: JavaScript, CSS, HTML - database: Not implemented
- libraries: Not implemented

MODULES: Note Management Module: This module is responsible for handling note-related functionality, including creating, editing, and deleting notes. It also handles file uploads and storage. Files: app.py

User Interface Module: This module is responsible for rendering the user interface, including HTML templates and CSS styling. Files: templates\index.html, static\css\style.css

File Upload Module: This module is responsible for handling file uploads and storing them in the uploads directory. Files: app.py

API ROUTES: - GET /notes: Retrieves a list of all notes. - POST /notes: Creates a new note. - GET /notes/:id: Retrieves a specific note by ID. - PUT /notes/:id: Updates a specific note by ID. - DELETE /notes/:id: Deletes a specific note by ID. - POST /upload: Uploads a file.

DATA FLOW: - When a user creates a new note, the Note Management Module receives the request and creates a new note in the database. The User Interface Module then renders the updated note list. - When a user retrieves a specific note, the Note Management Module retrieves the note from the database and returns it to the User Interface Module, which renders the note.

ARCHITECTURE: NoteNest uses a simple client-server architecture, with the backend written in Python and the frontend using JavaScript, CSS, and HTML. The system does not use a database, instead relying on file storage for note data.

SECURITY: - Not implemented in code

IMPROVEMENTS: - Implement user authentication and authorization to restrict access to notes. - Use a database to store note data for better scalability and reliability. - Implement data validation and sanitization to prevent SQL injection and cross-site scripting attacks.